

Multimodal Text-to-Image Application

Multimodal Text-to-Image Application

- 1. Concept Introduction
 - 1.1 What is Text-to-Image?
 - Core Principles
 - 1.2 What is FastSDCPU?
 - Core Features
 - Applicable Scenarios
- 2. Project Deployment
 - 2.1 Deployment Environment
 - 2.2 LAN Access
 - 2.3 Using Text-to-Image Functionality
 - 2.4 Changing Model Download Location

Since ollama does not support text-to-image functionality, we need to use other tools to implement local text-to-image. Currently, voice-controlled text-to-image is not supported, so the content of this article is the same as the offline text multimodal text-to-image application.

1. Concept Introduction

1.1 What is Text-to-Image?

Text-to-Image is an artificial intelligence technology that automatically generates corresponding images through **text descriptions**. Simply input a piece of text (for example, "a Shiba Inu wearing sunglasses surfing on the beach"), and the AI model will generate an image matching the description based on semantic understanding, without requiring any drawing or design skills.

Core Principles

1. **Language Understanding:** AI first parses your text, identifying keywords (such as "Shiba Inu", "sunglasses", "beach").
2. **Image Generation:** Based on training with massive image data, AI converts text into visual elements and combines them into a reasonable scene.
3. **Style Adaptation:** You can specify a style (realistic, cartoon, watercolor, etc.), and AI will adjust the generation effect accordingly.

1.2 What is FastSDCPU?

FastSDCPU is an open-source project specifically optimized for **CPU devices** for **Stable Diffusion text-to-image**. Through algorithm and engineering optimization, it achieves fast generation of high-quality images on ordinary computers without GPUs, significantly lowering the hardware barrier for AI art.

Core Features

1. **Pure CPU Operation:** No independent graphics card required; can run directly on laptops, desktops, or edge devices.
2. **High-Speed Inference:** Combined with Latent Consistency Models (LCM) and other technologies, images can be generated in just 4-8 steps, greatly reducing wait time.
3. **Lightweight Deployment:** Supports model quantization (such as INT8) and OpenVINO acceleration, reducing resource usage and improving response efficiency.

Applicable Scenarios

- Localized AI creative tools
- Enterprise intranet image generation services
- Teaching demonstrations and prototype development
- Lightweight deployment in resource-constrained environments

The Yahboom tutorial attachment provides a FastSDCPU compressed package. Users can choose to pull it to the image for decompression and use. After decompression, you can directly skip to Section 2.4 Startup Method After Successful Deployment for learning.

2. Project Deployment

2.1 Deployment Environment

Open a terminal and execute the following code:

```
#If git is not installed on the board, run these first
sudo apt update
sudo apt install git -y
sudo apt install python3.10-venv -y

#Add environment variables
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Clone project code
git clone https://github.com/rupehs/fastsdcpu.git
cd fastsdcpu

# Create virtual environment and install dependencies
python -m venv venv
source venv/bin/activate

#Install uv
curl -Ls https://astral.sh/uv/install.sh | sh(This step may fail without a proxy
in China. If it fails, skip this step and run the next command)

#If the curl command to install uv fails in the previous step, run these three
commands
wget https://mirrors.huaweicloud.com/astral/uv/0.8.4/uv-aarch64-unknown-linux-gnu
-O ~/.local/bin/uv
chmod +x ~/.local/bin/uv
~/.local/bin/uv --version

chmod +x install.sh start-webui.sh
./install.sh --disable-gui
```

Installation successful, press any key to exit:

```
+ tomesd==0.1.3
+ tomli==2.2.1
+ tomlikit==0.12.0
- torch==2.7.1+cpu
+ torch==2.5.1
+ torchvision==0.20.1
+ tqdm==4.67.1
+ transformers==4.48.0
+ typer==0.16.0
- typing-extensions==4.12.2
+ typing-extensions==4.8.0
+ tzdata==2025.2
+ urllib3==2.5.0
+ uvicorn==0.35.0
+ websockets==12.0
+ xxhash==3.5.0
+ yarl==1.20.1
+ zipp==3.23.0
FastSD CPU installation completed,press any key to continue...
```

2.2 LAN Access

Before starting, you need to modify a file to support LAN access, otherwise you can only access webui locally:

```
vim ~/fastsdcpu/src/frontend/webui/ui.py
```

After opening the ui.py file, scroll to the last line, find **webui.launch(share=share)**, and modify it to **webui.launch(server_name="0.0.0.0",share=share)**

Then save

Start:

```
./start-webui.sh
```

```
Found 7 stable diffusion models in config/stable-diffusion-models.txt
Found 4 LCM-LoRA models in config/lcm-lora-models.txt
Found 10 OpenVINO LCM models in config/openvino-lcm-models.txt
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_5m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_5m_224. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_11m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_11m_224. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_21m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_21m_224. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_21m_384 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_21m_384. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_21m_512 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_21m_512. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
Starting web UI mode
No lora models found, please add lora models to /home/sunrise/fastsdcpu/lora_models directory
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/gradio/components/dropdown.py:226: UserWarning: The value passed into gr.Dropdown() is not in the list of choices. Please update the list of choices to include: or set allow_custom_value=True.
    warnings.warn(
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/gradio/components/base.py:194: UserWarning: show_label has no effect when container is False.
    warnings.warn("show_label has no effect when container is False.")
* Running on local URL: http://0.0.0.0:7860

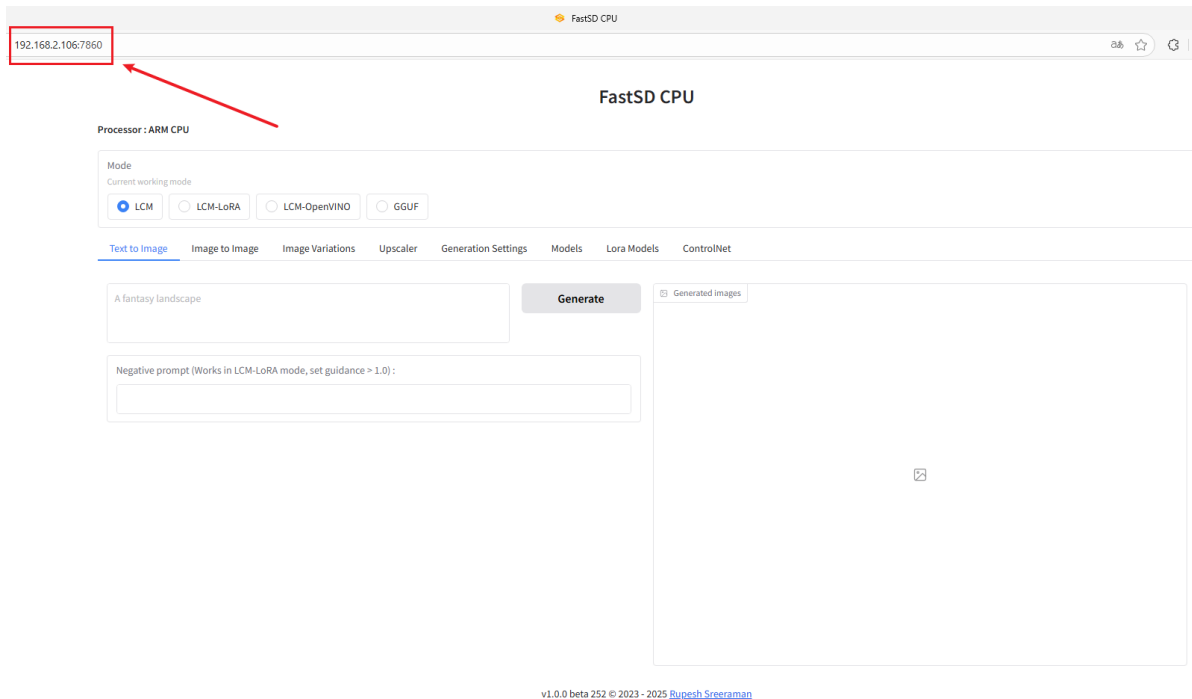
To create a public link, set `share=True` in `launch()`.
█
```

Then you can enter your board IP:7860 in the browser to access this webui.

2.3 Using Text-to-Image Functionality

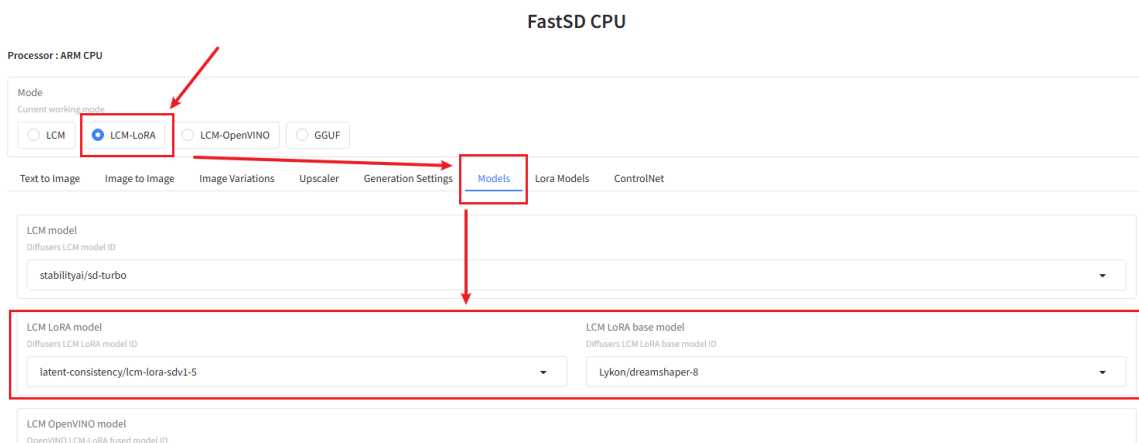
Use ifconfig in the terminal to check your board's IP, for example mine is 192.168.2.106.

Then open a browser and enter **your board IP:7860**. For example, I would enter 192.168.2.106:7860, and then you can access the webui.

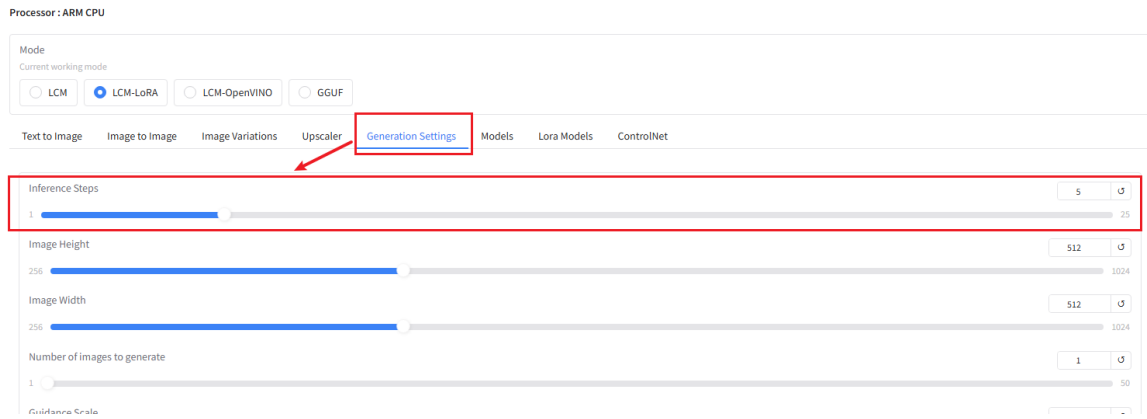


Then click on LCM-LoRA, which has relatively low memory usage. If you want to use other models, you can research on your own.

Then click on Models to see the LCM LoRA model settings. You can change to your desired model, or just choose the default like me.

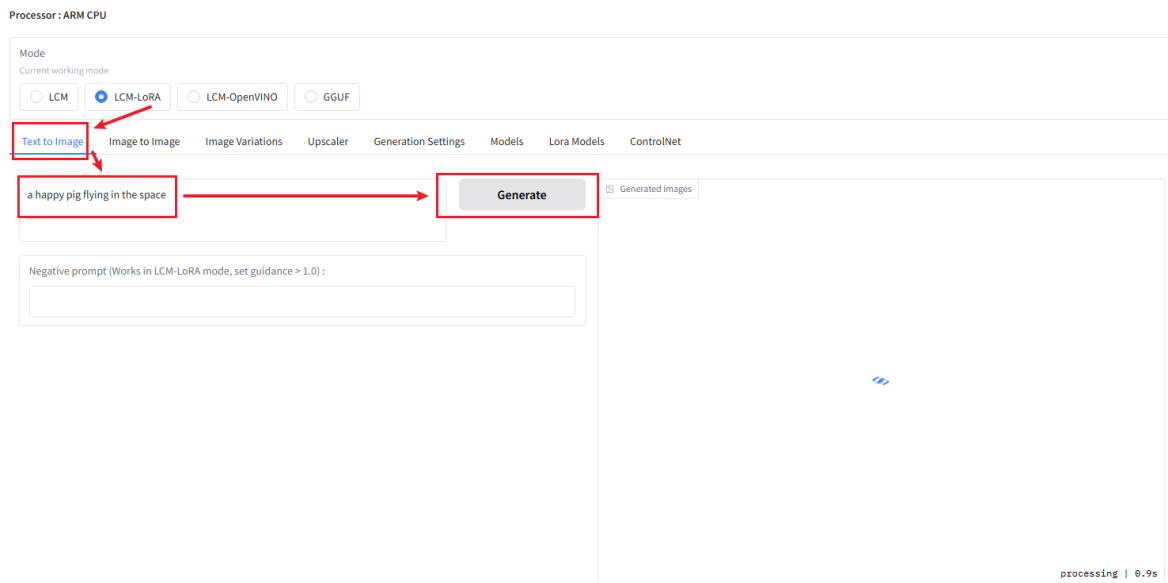


Then click on Generation Settings and increase the Inference Steps, which can improve the quality of generated images. I set it to 5 here.

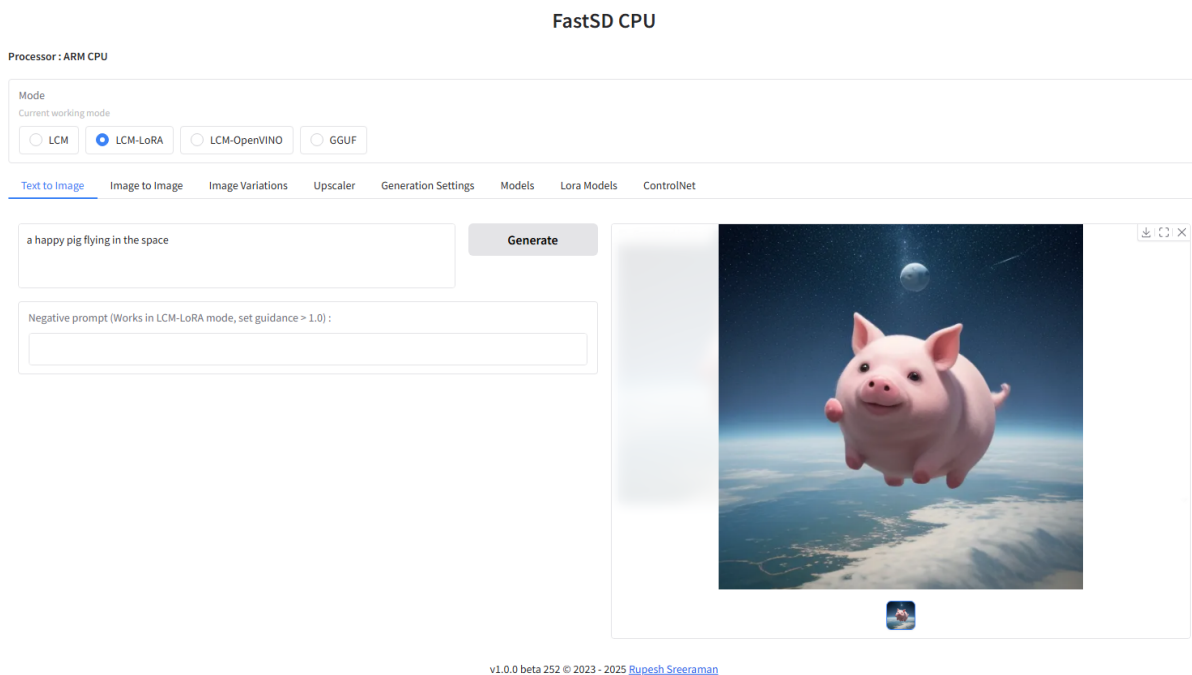


Then continue back to our Text to Image, enter what you want to generate in the dialog box, and press Generate to start generating images.

FastSD CPU



For first-time use, you need to download the model. You can see in the terminal that the default selected model is being downloaded. After the download is complete, the text-to-image function will begin executing.



2.4 Changing Model Download Location

You can use the following command in the terminal before each model download:

This will change your huggingface model download path. For example, if I want to place models on an SSD and not occupy board hard drive space, I can enter the following command:

```
export HF_HOME=/media/ssd/HF_Model
```

The /media/ssd path here is my mounted SSD path. You should modify it according to your own situation.

If you have already downloaded models to the default path and want to change the model location later, you can use the following command:

```
mv ~/.cache/huggingface /path/to/cache/directory #Change the path to your desired location
```

If you find it troublesome to use the command every time you download a model, you can directly add this export command to the .bashrc file.